

# Resolução do Quebra-Cabeça Tango

Busca Exaustiva e Backtracking aplicados a um  
Problema de Satisfação de Restrições

Arlindo Sérgio Pereira Júnior  
Arthur Astolfi Cardoso  
Guilherme de Almeida Rocha Vieira  
Joaquim Guilherme de Carvalho Vilela Silva  
Vitor Costa Vianna

## Resumo

Este trabalho apresenta a modelagem e a resolução do quebra-cabeça lógico **Tango**, um Problema de Satisfação de Restrições (CSP), por meio de duas estratégias clássicas: **Força Bruta** e **Backtracking**. O tabuleiro é representado como uma matriz de células de dois símbolos (Sol e Lua) sujeita a cinco restrições: preenchimento completo, limite de adjacência, equilíbrio de linhas e colunas, e as relações de igualdade (=) e oposição ( $\times$ ). A Força Bruta enumera as  $2^k$  combinações das  $k$  células vazias e valida cada estado completo, enquanto o Backtracking constrói a solução de forma incremental, aplicando as regras como *podas* a cada célula preenchida. Foram avaliados nove tabuleiros, em três tamanhos ( $4 \times 4$ ,  $6 \times 6$ ,  $8 \times 8$ ) e três níveis de dificuldade. Os experimentos mostram que a Força Bruta termina nos casos menores, mas explode rapidamente — no  $6 \times 6$  difícil ela ultrapassa 60s testando bilhões de combinações. Já o Backtracking resolve todos os nove tabuleiros em milissegundos, incluindo os  $8 \times 8$ , cujo espaço teórico chega a  $2^{58} \approx 2,9 \times 10^{17}$  estados, explorando de algumas dezenas a poucas centenas de nós.

## Sumário

|          |                                                               |          |
|----------|---------------------------------------------------------------|----------|
| <b>1</b> | <b>Modelagem do Problema</b>                                  | <b>2</b> |
| 1.1      | Formato do arquivo de entrada . . . . .                       | 2        |
| 1.2      | Separação das regras e validação . . . . .                    | 3        |
| 1.3      | Organização do código . . . . .                               | 3        |
| <b>2</b> | <b>Estratégia de Resolução por Força Bruta</b>                | <b>3</b> |
| 2.1      | Geração do espaço de estados . . . . .                        | 4        |
| 2.2      | Implementação . . . . .                                       | 4        |
| 2.3      | Validação aplicada . . . . .                                  | 4        |
| <b>3</b> | <b>Estratégia de Resolução por Backtracking</b>               | <b>5</b> |
| 3.1      | Função recursiva e condição de parada . . . . .               | 5        |
| 3.2      | As cinco regras como critérios de poda . . . . .              | 6        |
| 3.3      | Sobre a escolha da próxima célula . . . . .                   | 6        |
| <b>4</b> | <b>Exemplos de Execução</b>                                   | <b>6</b> |
| 4.1      | Exemplo 1 — 4x4 difícil: Força Bruta e Backtracking . . . . . | 7        |
| 4.2      | Exemplo 2 — 6x6 médio com Backtracking . . . . .              | 7        |
| 4.3      | Exemplo 3 — 8x8 difícil com Backtracking . . . . .            | 8        |

|          |                                |          |
|----------|--------------------------------|----------|
| <b>5</b> | <b>Análise de Complexidade</b> | <b>8</b> |
| 5.1      | Força Bruta . . . . .          | 8        |
| 5.2      | Backtracking . . . . .         | 9        |
| 5.3      | Conclusão . . . . .            | 10       |

## 1 Modelagem do Problema

O sistema foi implementado em Java. A ideia central foi representar o tabuleiro como uma matriz de células, em que cada célula assume um de três valores possíveis: Sol, Lua ou vazio. O projeto é organizado de forma modular, separando claramente a *lógica de validação das regras* (as podas) da *mecânica de recursão* do backtracking, conforme exigido pelo enunciado. A Tabela 1 resume as classes e suas responsabilidades.

Tabela 1: Principais classes e responsabilidades.

| Classe             | Responsabilidade                                                                                        |
|--------------------|---------------------------------------------------------------------------------------------------------|
| Board              | Matriz de células, tamanho do tabuleiro e lista de restrições.                                          |
| Cell               | Linha, coluna, valor e marca de <i>fixa</i> (dica inicial).                                             |
| CellValue          | Enum dos valores: SUN, MOON, EMPTY.                                                                     |
| Constraint         | Par de células e o tipo da restrição entre elas.                                                        |
| ConstraintType     | Enum: EQUAL (=) ou OPPOSITE (×).                                                                        |
| Rule               | Interface comum das regras (validação parcial e completa).                                              |
| AdjacencyRule      | Impede três símbolos iguais seguidos (horizontal/vertical).                                             |
| BalanceRule        | Garante igual número de sóis e luas por linha e coluna.                                                 |
| ConstraintRule     | Verifica as relações de igualdade e oposição.                                                           |
| RuleValidator      | Agrega as regras; oferece validação parcial e completa.                                                 |
| BoardReader        | Lê o arquivo de entrada e monta o Board.                                                                |
| BoardPrinter       | Imprime o tabuleiro e os sinais de restrição no terminal.                                               |
| ISolver            | Interface comum dos solvers ( <code>solve</code> , <code>getName</code> , <code>getStatistics</code> ). |
| SolverFactory      | Cria o(s) solver(s) conforme o modo escolhido.                                                          |
| BruteForceSolver   | Busca exaustiva sobre as $2^k$ combinações.                                                             |
| BacktrackingSolver | Busca recursiva com poda a cada célula preenchida.                                                      |

Cada célula (`Cell`) guarda sua posição, seu valor e uma marca de *fixa*: as dicas que já vêm no arquivo não podem ser alteradas pelos algoritmos, pois o método de escrita bloqueia células fixas. Assim, os solvers operam apenas sobre as posições originalmente vazias. As restrições = e × são objetos `Constraint`, cada um com as coordenadas de duas células vizinhas e o tipo (`EQUAL` ou `OPPOSITE`); essa representação por lista de pares permite consultar rapidamente apenas as restrições ligadas a uma célula recém-preenchida.

### 1.1 Formato do arquivo de entrada

A leitura é feita pela classe `BoardReader`. O arquivo começa com o tamanho do tabuleiro, seguido da grade inicial e, por fim, das restrições. A Tabela 2 descreve os símbolos usados.

Tabela 2: Símbolos do arquivo de entrada.

| Símbolo | Significado                                                       |
|---------|-------------------------------------------------------------------|
| S       | Sol (célula fixa)                                                 |
| L       | Lua (célula fixa)                                                 |
| .       | Célula vazia                                                      |
| H r c = | Restrição horizontal entre $(r, c)$ e $(r, c+1)$ , tipo igualdade |
| V r c X | Restrição vertical entre $(r, c)$ e $(r+1, c)$ , tipo oposição    |

## 1.2 Separação das regras e validação

As regras do jogo foram isoladas em classes próprias que implementam a interface `Rule`. Essa separação é o que permite reaproveitar exatamente as mesmas validações tanto na Força Bruta quanto no Backtracking, sem duplicação de código. A classe `RuleValidator` agrega as três regras e oferece dois modos:

- **Validação parcial** (`isValidPlacement`): usada durante o backtracking. Avalia apenas a linha e a coluna da célula recém-preenchida e as restrições ligadas a ela — o suficiente para podar caminhos inválidos sem reavaliar o tabuleiro inteiro.
- **Validação completa** (`isBoardValid`): usada quando o tabuleiro já está totalmente preenchido. Percorre todas as linhas, colunas e restrições e confirma a correteza global da solução.

No programa principal, o arquivo é lido novamente antes de executar cada algoritmo. Como ambos alteram o tabuleiro em memória durante a busca, reler garante que os dois métodos partam do mesmo estado inicial, tornando a comparação justa.

## 1.3 Organização do código

O código é dividido em pacotes por responsabilidade, o que mantém a lógica das regras (as podas) desacoplada da mecânica de busca:

```
src/  
  Main.java      -- ponto de entrada: leitura, execucao e impressao  
  model/  
    enums)      -- entidades do dominio (Board, Cell, Constraint,  
  rules/         -- interface Rule + as 3 regras do jogo  
  validation/    -- RuleValidator: compoe e aplica as regras  
  solver/        -- ISolver, SolverFactory, BruteForce/Backtracking  
  io/            -- BoardReader (entrada) e BoardPrinter (saida)
```

Essa estrutura apoia-se em alguns padrões de projeto que tornam o sistema extensível:

- **Interface Rule + composição.** Cada regra é uma classe isolada, e `RuleValidator` as combina percorrendo uma lista. Acrescentar uma nova regra significa criar uma classe nova, sem alterar os solvers — a validação fica totalmente separada da busca.
- **Interface ISolver (Strategy).** Força Bruta e Backtracking expõem o mesmo contrato (`solve`, `getName`, `getStatistics`), de modo que o `Main` os executa de forma uniforme, sem conhecer detalhes de cada algoritmo.
- **SolverFactory (Factory).** Centraliza a escolha do(s) algoritmo(s) a partir do modo (`bruteforce`, `backtracking` ou `ambos`), eliminando `if/else` espalhados pelo `Main` e isolando a decisão em um único ponto.

Na prática, o `Main` apenas lê o tabuleiro, pede os solvers à fábrica e, para cada um, cronometra a chamada a `solve` e imprime as estatísticas — toda a complexidade das regras e dos algoritmos fica encapsulada nos respectivos pacotes.

## 2 Estratégia de Resolução por Força Bruta

A *força bruta* (ou busca exaustiva) é o paradigma mais direto de resolução de um problema de busca: enumerar *todos* os candidatos possíveis e testar cada um contra as condições do problema, até achar uma solução ou esgotar o espaço. É a abordagem clássica de *gerar e testar* — primeiro produz-se um estado completo, depois verifica-se sua validade. Suas virtudes são a simplicidade e a *completeude*: se existe solução, ela será encontrada. Sua fraqueza é o custo, pois o espaço de

candidatos costuma crescer exponencialmente. No contexto de um problema que precisa satisfazer restrições, equivale a atribuir um valor a *todas* as variáveis de uma só vez e só então conferir as restrições.

Na Força Bruta o algoritmo não tenta ser inteligente durante o preenchimento. Ele primeiro identifica todas as células vazias e depois gera, uma a uma, todas as combinações possíveis de Sol e Lua para essas células, validando o tabuleiro *somente quando ele está completo*. Esse é o ponto que diferencia a Força Bruta do Backtracking: aqui a verificação das regras ocorre apenas no estado final, nunca durante a construção.

## 2.1 Geração do espaço de estados

Como cada célula vazia admite apenas dois valores, se existirem  $k$  células vazias o total de combinações é exatamente

$$2^k.$$

No código (classe `BruteForceSolver`), as posições vazias são guardadas em uma lista e cada combinação é representada por uma **máscara de bits** de  $k$  bits: o bit  $i$  em 0 coloca Sol na  $i$ -ésima célula vazia, e em 1 coloca Lua. Percorrendo a máscara de 0 até  $2^k - 1$ , enumeramos todo o espaço de estados. Por exemplo, com 4 células vazias testam-se as combinações 0000 a 1111, totalizando 16 estados.

## 2.2 Implementação

O trecho a seguir é o núcleo da classe `BruteForceSolver`. O método `solve` itera sobre todas as máscaras e só chama `isBoardValid` quando o tabuleiro está inteiramente preenchido; `fill` traduz cada máscara em uma atribuição de Sol/Lua às células vazias.

```
public boolean solve(Board board) {
    List<int[]> emptyCells = collectEmptyCells(board);
    int n = emptyCells.size();
    totalCombinations = 1L << n;           // espaco total = 2^n
    for (long mask = 0; mask < totalCombinations; mask++) {
        fill(board, emptyCells, mask);
        combinationsTried++;
        if (validator.isBoardValid(board)) // valida o tabuleiro
            COMPLETO
            return true;
    }
    return false;
}

private void fill(Board board, List<int[]> cells, long mask) {
    for (int i = 0; i < cells.size(); i++) {
        int[] pos = cells.get(i);
        CellValue val = ((mask >> i) & 1) == 0 ? CellValue.SUN :
            CellValue.MOON;
        board.setCell(pos[0], pos[1], val);
    }
}
```

## 2.3 Validação aplicada

Depois que uma combinação completa é colocada, o validador confere todas as regras de uma só vez sobre o tabuleiro inteiro:

- todas as células estão preenchidas;

- não há três símbolos iguais seguidos (adjacência);
- cada linha tem o mesmo número de sóis e luas (equilíbrio);
- cada coluna tem o mesmo número de sóis e luas (equilíbrio);
- todas as restrições = e  $\times$  estão satisfeitas.

Se a combinação é válida, o algoritmo termina; caso contrário, passa para a próxima máscara. Em um caso com solução, ele não percorre necessariamente todo o espaço — só faria isso se a solução estivesse na última combinação ou se não houvesse solução. Como cada validação completa percorre linhas, colunas e restrições, seu custo é  $O(n^2)$  para um tabuleiro  $n \times n$ ; multiplicado pelo número de estados, o custo total da Força Bruta é  $O(2^k \cdot n^2)$ .

Essa estratégia é viável apenas em tabuleiros pequenos, como o  $4 \times 4$ . Em tabuleiros maiores o número de combinações explode (Seção 5), e por isso a Força Bruta foi usada principalmente para evidenciar o espaço total de busca e servir de referência para comparar com o Backtracking.

### 3 Estratégia de Resolução por Backtracking

O *backtracking* é um refinamento da busca exaustiva que constrói a solução de forma *incremental*, uma variável por vez, verificando as restrições a cada passo. Assim que uma atribuição parcial viola alguma restrição, o algoritmo *retrocede* (*backtrack*): desfaz a última escolha e tenta outra, sem nunca explorar as combinações que partiriam daquele estado já inválido. É, em essência, uma busca em profundidade (DFS) sobre a árvore de estados, com *poda* (*pruning*) dos ramos inviáveis — e é o algoritmo clássico para problemas de satisfação de restrições, justamente porque as restrições do problema servem de teste de consistência que elimina cedo grandes regiões do espaço. A diferença essencial em relação à força bruta está em *quando* se valida: aqui, durante a construção; lá, apenas no estado final.

O Backtracking também preenche o tabuleiro por tentativa, mas não espera o estado ficar completo para validar. A cada valor colocado ele verifica se aquela escolha *ainda pode* levar a uma solução válida; em caso negativo, abandona o caminho imediatamente (poda) e tenta outro valor. É essa antecipação da validação que evita a explosão combinatória da Força Bruta.

#### 3.1 Função recursiva e condição de parada

A função principal (classe `BacktrackingSolver`) procura a próxima célula vazia varrendo a matriz de cima para baixo e da esquerda para a direita. Para cada célula tenta primeiro Sol e depois Lua. A **condição de parada (caso base)** ocorre quando não há mais nenhuma célula vazia: o tabuleiro está completo e faz-se a validação final. Se ela passa, a solução foi encontrada.

O trecho a seguir é a função recursiva da classe `BacktrackingSolver` (os incrementos dos contadores de estatística foram omitidos para clareza):

```
private boolean backtrack(Board board) {
    int[] next = findNextEmpty(board);
    if (next == null) { // CASO BASE: tabuleiro
        completo
        return validator.isBoardValid(board);
    }
    int row = next[0], col = next[1];
    for (CellValue candidate : new CellValue[]{CellValue.SUN, CellValue.MOON}) {
        board.setCell(row, col, candidate);
        if (validator.isValidPlacement(board, row, col)) { // PODA: so
            o afetado
            if (backtrack(board)) return true;
        }
    }
}
```

```

        board.setCell(row, col, CellValue.EMPTY); //
        RETROCESSO
    }
    return false;
}

```

Quando uma tentativa não serve, o algoritmo desfaz a escolha (volta a célula para vazia) e tenta o próximo valor — esse é o *retrocesso*. O código mantém contadores de nós explorados e de retrocessos para permitir a comparação quantitativa com a Força Bruta (Seção 4).

### 3.2 As cinco regras como critérios de poda

A poda é o coração da implementação. A validação parcial avalia apenas a parte do tabuleiro afetada pela última escolha — a linha e a coluna da célula preenchida e as restrições ligadas a ela —, o que evita reavaliar o tabuleiro inteiro a cada nível da recursão. A Tabela 3 associa cada uma das cinco regras do enunciado ao momento em que ela atua como poda e à classe responsável.

Tabela 3: As cinco regras do Tango e seu papel como poda no backtracking.

| Regra (enunciado)               | Papel na busca                                                                | Classe         |
|---------------------------------|-------------------------------------------------------------------------------|----------------|
| 1. Preenchimento completo       | Condição de parada: dispara a validação final quando não há células vazias.   | RuleValidator  |
| 2. Limite de adjacência         | Poda: corta o ramo se surgirem três símbolos iguais seguidos na linha/coluna. | AdjacencyRule  |
| 3. Equilíbrio de linhas/colunas | Poda: corta se a linha ou coluna ultrapassar $n/2$ sóis ou $n/2$ luas.        | BalanceRule    |
| 4. Restrição de igualdade (=)   | Poda: corta se duas células ligadas por = já tiverem valores diferentes.      | ConstraintRule |
| 5. Restrição de oposição (×)    | Poda: corta se duas células ligadas por × já tiverem valores iguais.          | ConstraintRule |

Vale destacar a poda de equilíbrio: na validação parcial ela usa o critério “no máximo  $n/2$ ” ( $\leq$ ), e não “exatamente  $n/2$ ” ( $=$ ), porque o tabuleiro ainda está incompleto. Já a validação final exige a igualdade exata. As restrições = e ×, por sua vez, são consideradas satisfeitas enquanto uma das duas células ainda estiver vazia, e só viram poda quando ambas têm valor.

### 3.3 Sobre a escolha da próxima célula

A varredura linha a linha é deliberadamente simples e já foi suficiente para os tabuleiros do trabalho. Uma heurística mais elaborada — por exemplo, escolher primeiro a célula mais restrita — reduziria algumas tentativas, mas tornaria o código menos legível. Como o objetivo era contrastar Força Bruta e Backtracking, preferimos manter a poda das regras como o ponto central da implementação.

## 4 Exemplos de Execução

O projeto é compilado e executado a partir do terminal:

```

javac -d out $(find src -name '*.java')
java -cp out Main <caminho-do-puzzle> [bruteforce|backtracking|ambos]

```

Para os experimentos foram preparados **nove tabuleiros**, em três tamanhos ( $4 \times 4$ ,  $6 \times 6$  e  $8 \times 8$ ) e três níveis de dificuldade (*fácil*, *médio* e *difícil*). Aqui, a dificuldade é definida pelo número de células vazias  $k$ : quanto menos dicas iniciais, maior o espaço de busca. A Força Bruta foi executada apenas nos tabuleiros  $4 \times 4$  e  $6 \times 6$  (nos  $8 \times 8$  ela é inviável), com um limite de tempo de 60 segundos por execução. O Backtracking foi executado em todos os nove.

**Ambiente de teste.** Todas as medições foram feitas em um **MacBook Air (Apple M4, 10 núcleos, 16 GB de RAM)**, com **macOS 15.7.3** e **OpenJDK 25**. Os tempos reportados são os retornados pelo próprio programa (método de estatísticas, que cronometra apenas a chamada ao solver). Como os *contadores* (combinações testadas, nós explorados e retrocessos) são independentes da máquina, eles continuam sendo a métrica principal; os tempos, em segundos, servem como referência concreta neste ambiente. A Tabela 4 consolida todos os resultados.

Tabela 4: Resumo das execuções. Legenda das colunas:  $k$  = células vazias; **Espaço  $2^k$**  = número total de estados possíveis; **FB test.** = combinações testadas pela Força Bruta; **FB t(s)** = tempo da Força Bruta em segundos; **BT nós** = nós explorados pelo Backtracking; **BT retr.** = retrocessos do Backtracking; **BT t(s)** = tempo do Backtracking em segundos. “*timeout*” = não terminou em 60s; “–” = Força Bruta não executada (inviável em  $8 \times 8$ ). Tempos medidos no ambiente descrito acima.

| Tabuleiro     | $k$ | Espaço $2^k$          | FB test.       | FB t(s) | BT nós | BT retr. | BT t(s) |
|---------------|-----|-----------------------|----------------|---------|--------|----------|---------|
| 4 × 4 fácil   | 6   | 64                    | 37             | 0,004   | 8      | 2        | 0,009   |
| 4 × 4 médio   | 9   | 512                   | 365            | 0,008   | 14     | 5        | 0,008   |
| 4 × 4 difícil | 12  | 4096                  | 2320           | 0,011   | 26     | 14       | 0,008   |
| 6 × 6 fácil   | 24  | $1,68 \times 10^7$    | 9632614        | 1,273   | 38     | 14       | 0,009   |
| 6 × 6 médio   | 28  | $2,68 \times 10^8$    | 170182234      | 27,449  | 108    | 80       | 0,009   |
| 6 × 6 difícil | 32  | $4,29 \times 10^9$    | <i>timeout</i> | –       | 266    | 234      | 0,012   |
| 8 × 8 fácil   | 44  | $1,76 \times 10^{13}$ | –              | –       | 143    | 99       | 0,010   |
| 8 × 8 médio   | 52  | $4,50 \times 10^{15}$ | –              | –       | 383    | 331      | 0,012   |
| 8 × 8 difícil | 58  | $2,88 \times 10^{17}$ | –              | –       | 147    | 89       | 0,012   |

Os tempos do Backtracking ( $\approx 0,01$  s) são dominados pela inicialização da JVM; a busca em si é praticamente instantânea em todos os casos. Dois fatos chamam a atenção na tabela. Primeiro, no  $6 \times 6$  a Força Bruta ainda termina nos níveis fácil (1,3s, testando  $\approx 9,6$  milhões de combinações) e médio (27,4s,  $\approx 170$  milhões), mas *estoura o limite* de 60s no difícil, cujo espaço chega a  $2^{32} \approx 4,3$  bilhões — evidência direta da explosão combinatória. Segundo, o  $8 \times 8$  *difícil* explora *menos* nós (147) que o *médio* (383), embora tenha mais células vazias: ele possui mais restrições  $=/\times$ , que podam o espaço com mais força. Ou seja, o esforço de busca depende não só do número de vazias, mas também da quantidade de informação fornecida pelas restrições.

#### 4.1 Exemplo 1 — 4x4 difícil: Força Bruta e Backtracking

```
java -cp out Main puzzles/hard_4x4.txt ambos
```

```
=== Tabuleiro Inicial ===
+-----+-----+
| . = . | S = S |
+-----+-----+
| . | . | . = . |
+-----+-----+
| . | . | L | . |
+-----+-----+
| . | . | . | L |
+-----+-----+
```

```
=== Tabuleiro Resolvido ===
+-----+-----+
| L = L | S = S |
+-----+-----+
| L | L | S = S |
+-----+-----+
| S | S | L | L |
+-----+-----+
| S | S | L | L |
+-----+-----+
```

**Força Bruta:** 2320 combinações testadas (de 4096). **Backtracking:** 26 nós, 14 retrocessos. Mesmo neste tabuleiro pequeno o Backtracking explora cerca de  $90\times$  menos estados que a Força Bruta.

#### 4.2 Exemplo 2 — 6x6 médio com Backtracking

```
java -cp out Main puzzles/medium_6x6.txt backtracking
```

```

=== Tabuleiro Inicial ===
+-----+-----+-----+-----+-----+-----+
| . x . | . | . | . x S |
+-----+-----+-----+-----+-----+
| . | . | . = . | . | . |
+-----+-----+-----+-----+-----+
| . | . | L | L x . | . |
+-----+-----+-----+-----+-----+
| L | . | . = . | . | . |
+-----+-----+-----+-----+-----+
| L | . | L = . x . | . |
+-----+-----+-----+-----+-----+
| . | L | . | . | S | . |
+-----+-----+-----+-----+-----+

```

```

=== Tabuleiro Resolvido ===
+-----+-----+-----+-----+-----+-----+
| L x S | S | L | L x S |
+-----+-----+-----+-----+-----+
| S | L | S = S | L | L |
+-----+-----+-----+-----+-----+
| S | S | L | L x S | L |
+-----+-----+-----+-----+-----+
| L | L | S = S | L | S |
+-----+-----+-----+-----+-----+
| L | S | L = L x S | S |
+-----+-----+-----+-----+-----+
| S | L | L | S | S | L |
+-----+-----+-----+-----+-----+

```

Estatísticas: Nos explorados : 108, Retrocessos : 80. Para o *mesmo* tabuleiro, a Força Bruta precisou de  $\approx 170$  milhões de combinações (27,4s) para chegar à solução, contra apenas 108 nós do Backtracking (resolvido em milissegundos).

### 4.3 Exemplo 3 — 8x8 difícil com Backtracking

```
java -cp out Main puzzles/hard_8x8.txt backtracking
```

```

=== Tabuleiro Resolvido ===
+-----+-----+-----+-----+-----+-----+
| S | S | L = L | S = S | L | L |
+-----+-----+-----+-----+-----+
| S x L | S = S x L x S | L | L |
+-----+-----+-----+-----+-----+
| L | S | L | L x S | L | S | S |
+-----+-----+-----+-----+-----+
| L | L | S = S | L = L | S = S |
+-----+-----+-----+-----+-----+
| S | S x L | S x L | S x L | L |
+-----+-----+-----+-----+-----+
| L | L | S | L x S | S | L | S |
+-----+-----+-----+-----+-----+
| L | S | S | L | S | L | S | L |
+-----+-----+-----+-----+-----+
| S | L | L | S | L | L | S | S |
+-----+-----+-----+-----+-----+

```

```

--- Backtracking ---
Nos explorados : 147
Retrocessos    : 89

```

Com 58 células vazias, o espaço teórico chega a  $2^{58} \approx 2,9 \times 10^{17}$  estados — impossível para a Força Bruta. O Backtracking, ainda assim, encontra a solução única explorando apenas 147 nós, graças às podas combinadas das regras de adjacência, equilíbrio e das numerosas restrições deste tabuleiro.

## 5 Análise de Complexidade

O fator determinante da complexidade do Tango é o número de células vazias  $k$ . Como cada uma admite dois valores, o espaço de estados tem tamanho

$$2^k.$$

### 5.1 Força Bruta

A Força Bruta percorre os  $2^k$  estados e, para cada um, executa a validação completa, que varre linhas, colunas e restrições a um custo de  $O(n^2)$  num tabuleiro  $n \times n$ . Logo, a complexidade de pior caso é

$$O(2^k \cdot n^2),$$



isto é, **exponencial** no número de células vazias. O termo  $2^k$  domina amplamente o fator polinomial  $n^2$ , de modo que o gargalo é a quantidade de combinações. Os experimentos confirmam esse crescimento: nos  $4 \times 4$  a busca é instantânea; no  $6 \times 6$  fácil ( $k = 24$ ) ela termina em  $\approx 1,3$  s, no médio ( $k = 28$ ) em  $\approx 27$  s, mas no difícil ( $k = 32$ , mais de 4 bilhões de estados) ela *não termina* dentro do limite de 60 s. Para os  $8 \times 8$ , com  $k$  entre 44 e 58, o espaço atinge  $2^{58} \approx 2,9 \times 10^{17}$  — totalmente impraticável.

## 5.2 Backtracking

O Backtracking compartilha o pior caso exponencial, mas na prática visita uma fração ínfima da árvore de busca, porque aplica as regras como poda logo após preencher cada célula. As podas empregadas foram:

- proibir três símbolos iguais consecutivos (adjacência);
- impedir que uma linha ou coluna passe de  $n/2$  sóis ou  $n/2$  luas (equilíbrio);
- rejeitar restrições = e  $\times$  já violadas assim que ambas as células envolvidas têm valor.

Cada poda elimina, de uma só vez, todas as combinações que partiriam daquele prefixo inválido. A Tabela 5 mostra a fração do espaço teórico efetivamente explorada pelo Backtracking em cada tabuleiro.

Tabela 5: Nós explorados pelo Backtracking frente ao espaço teórico  $2^k$ .

| Tabuleiro            | Espaço $2^k$         | BT nós | Fração explorada      |
|----------------------|----------------------|--------|-----------------------|
| $4 \times 4$ fácil   | $6,4 \times 10^1$    | 8      | $1,3 \times 10^{-1}$  |
| $4 \times 4$ médio   | $5,1 \times 10^2$    | 14     | $2,7 \times 10^{-2}$  |
| $4 \times 4$ difícil | $4,1 \times 10^3$    | 26     | $6,3 \times 10^{-3}$  |
| $6 \times 6$ fácil   | $1,7 \times 10^7$    | 38     | $2,3 \times 10^{-6}$  |
| $6 \times 6$ médio   | $2,7 \times 10^8$    | 108    | $4,0 \times 10^{-7}$  |
| $6 \times 6$ difícil | $4,3 \times 10^9$    | 266    | $6,2 \times 10^{-8}$  |
| $8 \times 8$ fácil   | $1,8 \times 10^{13}$ | 143    | $8,1 \times 10^{-12}$ |
| $8 \times 8$ médio   | $4,5 \times 10^{15}$ | 383    | $8,5 \times 10^{-14}$ |
| $8 \times 8$ difícil | $2,9 \times 10^{17}$ | 147    | $5,1 \times 10^{-16}$ |

A fração explorada despenca à medida que o tabuleiro cresce: de 13% no  $4 \times 4$  fácil para cerca de  $10^{-16}$  no  $8 \times 8$  difícil. Vale notar que o  $8 \times 8$  difícil explora *menos* nós (147) que o médio (383): por ter mais restrições =/ $\times$ , suas podas são mais agressivas. Isso reforça que o custo real do Backtracking depende tanto do número de vazias quanto da informação trazida pelas restrições, e não apenas do tamanho bruto do espaço.

A Figura 1 ilustra, em escala logarítmica, o abismo entre o espaço teórico  $2^k$  (que a Força Bruta teria de percorrer) e os poucos nós visitados pelo Backtracking nos nove tabuleiros.

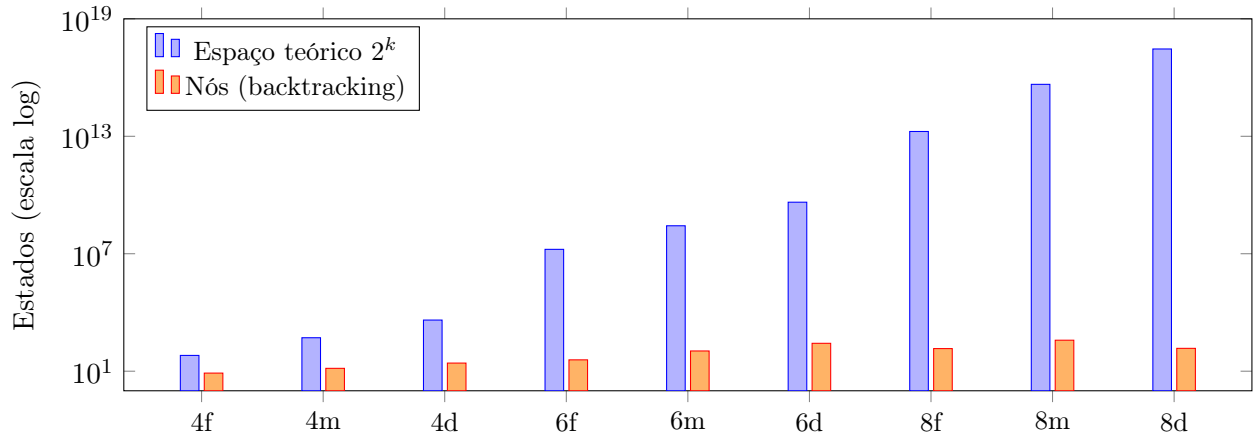


Figura 1: Espaço de busca teórico versus nós explorados pelo Backtracking nos nove tabuleiros (4/6/8 = tamanho; f/m/d = fácil/médio/difícil).

### 5.3 Conclusão

Este trabalho modelou o Tango como um Problema de Satisfação de Restrições e o resolveu por duas estratégias complementares, evidenciando na prática a diferença entre busca exaustiva e busca com poda.

A Força Bruta cumpriu seu papel de tornar explícita a **explosão combinatória**: ela resolve os  $4 \times 4$  e os  $6 \times 6$  mais simples, mas seu custo cresce de forma incontrolável — de 1,3s no  $6 \times 6$  fácil para 27,4s no médio, até *não terminar* no difícil ( $2^{32}$  estados) — e torna-se impensável nos  $8 \times 8$ , onde o espaço chega a  $2^{58} \approx 2,9 \times 10^{17}$ . Esse comportamento confirma empiricamente a complexidade  $O(2^k \cdot n^2)$  deduzida na análise.

O Backtracking, por outro lado, mostrou que a chave para domar esse espaço não é maior poder de processamento, mas **usar as próprias regras do problema como critérios de poda**. Validando adjacência, equilíbrio e as restrições  $=/\times$  logo após cada célula, ele descarta prefixos inválidos antes de explorá-los e, com isso, visita apenas uma fração ínfima do espaço — de 13% no menor tabuleiro a cerca de  $10^{-16}$  no maior. O resultado é que os *nove* tabuleiros, inclusive os  $8 \times 8$ , são resolvidos em milissegundos.

Os experimentos também revelaram um ponto sutil mas importante: o esforço de busca não depende apenas do tamanho do tabuleiro, mas da **informação disponível**. O  $8 \times 8$  difícil, apesar de ter mais células vazias, explorou menos nós que o médio, porque suas restrições adicionais podam o espaço mais cedo. Isso reforça a natureza de CSP do problema: cada restrição não é só uma condição a verificar, mas uma ferramenta ativa de redução do espaço de busca.

Em síntese, a Força Bruta é didaticamente valiosa por expor a dimensão real do problema, mas é o Backtracking — pela combinação de recursão incremental e podas baseadas nas regras do jogo — a estratégia efetivamente adequada para resolver o Tango na prática.

## Referências

- [1] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022. Cap. 34 (NP-Completeness) e Apêndice sobre backtracking.